

Modules in Python

What Are Modules?

Libraries of code containing functions for you to use

Why Do We Use Modules?

No need to reinvent the wheel

Functions are written, optimized, tested

Decrease code size and complexity

Update/fix one place instead of many

Common Modules For Data Scientists

Pandas

NumPy

Matplotlib

Making A Module

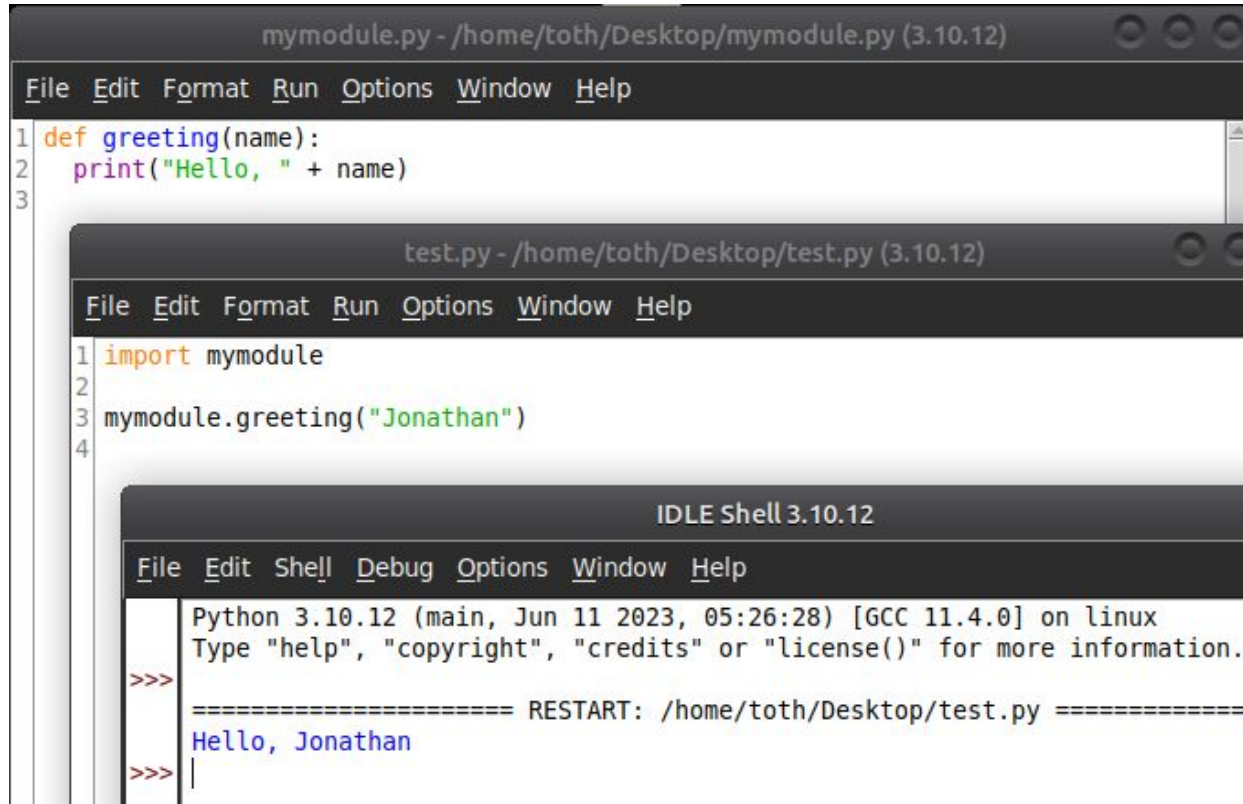
Save your code in a .py file

Using A Module

Import module

Call the function you need

Example



The screenshot displays the Python IDLE environment with three windows. The top window, titled 'mymodule.py - /home/toth/Desktop/mymodule.py (3.10.12)', contains a function definition: `def greeting(name):` followed by `print("Hello, " + name)` on the next line. The second window, titled 'test.py - /home/toth/Desktop/test.py (3.10.12)', shows the import and execution: `import mymodule` followed by `mymodule.greeting("Jonathan")`. The third window, titled 'IDLE Shell 3.10.12', shows the command prompt output: `>>>` followed by the shell's startup message, then a restart message for 'test.py', and finally the output 'Hello, Jonathan'.

```
mymodule.py - /home/toth/Desktop/mymodule.py (3.10.12)
File Edit Format Run Options Window Help
1 def greeting(name):
2   print("Hello, " + name)
3

test.py - /home/toth/Desktop/test.py (3.10.12)
File Edit Format Run Options Window Help
1 import mymodule
2
3 mymodule.greeting("Jonathan")
4

IDLE Shell 3.10.12
File Edit Shell Debug Options Window Help
Python 3.10.12 (main, Jun 11 2023, 05:26:28) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: /home/toth/Desktop/test.py =====
Hello, Jonathan
>>> |
```

Common Usage

Example

Load a CSV file into a Pandas DataFrame:

```
import pandas as pd  
  
df = pd.read_csv('data.csv')  
  
print(df.to_string())
```

Import the module as something

Call the function using the something.function_name

Example

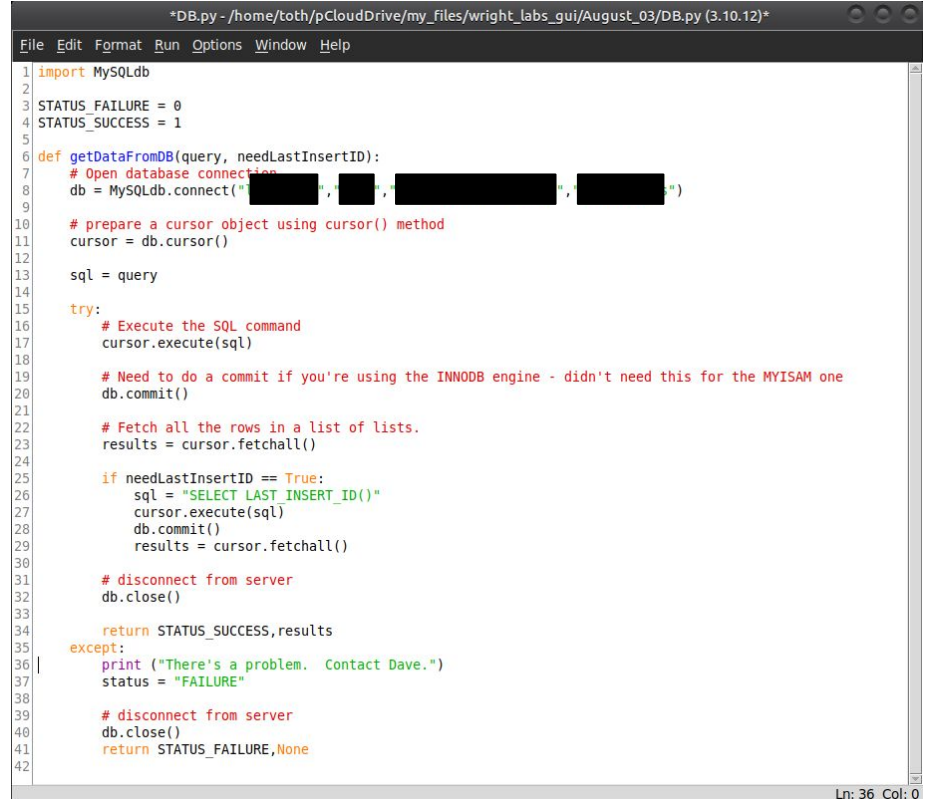
Create a NumPy array:

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5])  
  
print(arr)
```

Why Do You Care?

Simplify your code, especially with database access!

```
23| import DB  
  
58| status, results = DB.getDataFromDB("SELECT * FROM Client", False)
```



```
*DB.py - /home/toth/pCloudDrive/my_files/wright_labs_gui/August_03/DB.py (3.10.12)*  
File Edit Format Run Options Window Help  
1 import MySQLdb  
2  
3 STATUS_FAILURE = 0  
4 STATUS_SUCCESS = 1  
5  
6 def getDataFromDB(query, needLastInsertID):  
7     # Open database connection  
8     db = MySQLdb.connect("localhost", "root", "password", "database")  
9  
10    # prepare a cursor object using cursor() method  
11    cursor = db.cursor()  
12  
13    sql = query  
14  
15    try:  
16        # Execute the SQL command  
17        cursor.execute(sql)  
18  
19        # Need to do a commit if you're using the INNODB engine - didn't need this for the MYISAM one  
20        db.commit()  
21  
22        # Fetch all the rows in a list of lists.  
23        results = cursor.fetchall()  
24  
25        if needLastInsertID == True:  
26            sql = "SELECT LAST_INSERT_ID()  
27            cursor.execute(sql)  
28            db.commit()  
29            results = cursor.fetchall()  
30  
31        # disconnect from server  
32        db.close()  
33  
34        return STATUS_SUCCESS, results  
35    except:  
36        print ("There's a problem. Contact Dave.")  
37        status = "FAILURE"  
38  
39        # disconnect from server  
40        db.close()  
41        return STATUS_FAILURE, None  
42
```